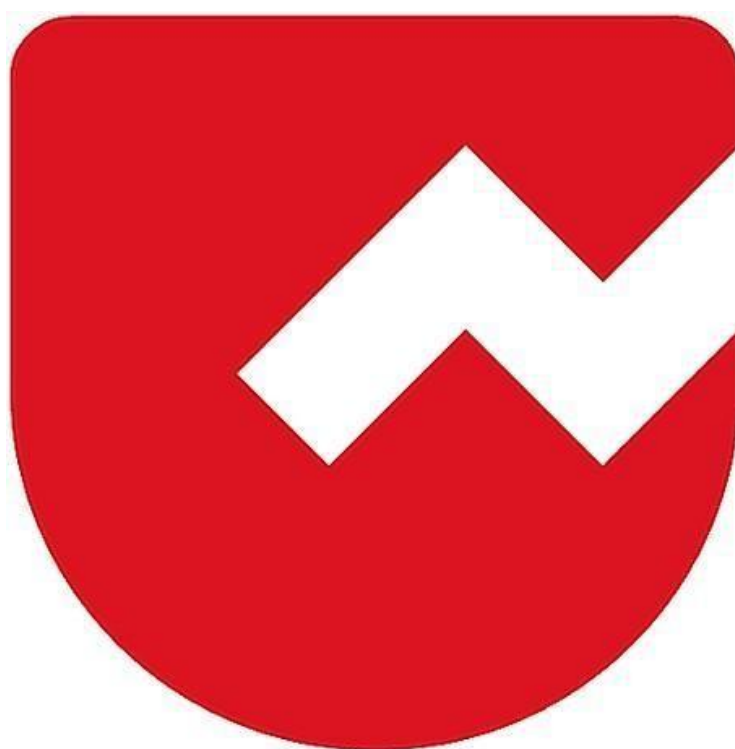




Zoho Schools for Graduate Studies



Notes

Abstract methods

Abstract Method

- A method without implementation or definition or body is termed as an abstract method.

Concrete Method

- A method with implementation or definition or well-defined body is termed as a concrete method. (or) a well-defined method is a concrete method.

Abstract Class

- A class that cannot be instantiated is an abstract class.

Concrete Class

- A class with only well-defined members is a concrete class.

Note:

- If a class contains at least one abstract member, then the class must be declared as abstract.
- A class can be marked as abstract even if it contains only concrete members.
- Can a class be marked as abstract even if it contains only concrete members?
yes

- There must exist some concrete sub-class to provide implementation for the inherited abstract members.

(or)

- The first concrete sub-class must provide implementation for all the inherited abstract methods.

Abstract Class: Vehicle

```
abstract public class Vehicle
{
    private String name;
    private double distance;
    private double fuelConsumed;

    Vehicle(String name, double distance, double fuelConsumed)
    {
        this.name = name;
        this.distance = distance;
        this.fuelConsumed = fuelConsumed;
    }

    // Getters and setters...

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }
}
```

```
public double getDistance()
{
    return distance;
}

public void setDistance(double distance)
{
    this.distance = distance;
}

public double getFuelConsumed()
{
    return fuelConsumed;
}

public void setFuelConsumed(double fuelConsumed)
{
    this.fuelConsumed = fuelConsumed;
}

    abstract public double calculateMileage();
}
```

Explanation :

- Vehicle is an **abstract class** (you cannot create objects of it directly).
- It has **common properties**:
 - name → brand name of vehicle
 - distance → distance travelled (in km)
 - fuelConsumed → how much fuel/Battery it used (in litres/Kilowatts)
- It has an **abstract method** calculateMileage() → each child class must define how to calculate mileage.

Child Class: Bike

```
class Bike extends Vehicle
{
    Bike(String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }

    public double calculateMileage()
    {
        return getDistance() / getFuelConsumed();
    }
}
```

Explanation:

- Bike **extends** Vehicle.
- In the constructor, it calls `super(...)` to set name, distance, and fuelConsumed.
- Implements `calculateMileage()` as **distance ÷ fuelConsumed** (km per litre).

Child Class: ElectricVehicle

```
class ElectricVehicle extends Vehicle
{
    ElectricVehicle (String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }
}
```

```
public double calculateMileage()
{
    return getDistance() / getFuelConsumed();
}
}
```

Explanation:

- ElectricVehicle extends Vehicle
- Right now, it does the same calculation as Bike.
- Idea: In future, if electric vehicles calculate mileage differently (say, distance per kWh), we could override logic here.

Child Class: ElectricVehicle

```
class Scooter extends Vehicle
{
    Scooter(String name, double distance, double fuelConsumed)
    {
        super(name, distance, fuelConsumed);
    }

    public double calculateMileage()
    {
        return getDistance() / getFuelConsumed();
    }
}
```

Explanation:

- Scooter extends Vehicle.
- Same calculation for now, but written separately for clarity.
- Shows how **multi-level inheritance** works.

Main Class

```
public class Main
{
    public static void main(String[] args)
    {
        Bike vobj1 = new Bike("Ducati", 40, 3.5);
        double bikeVal = vobj1.calculateMileage();

        Scooter vobj3 = new Scooter("Honda", 35, 8.3);
        double scootVal = vobj3.calculateMileage();

        System.out.println("Brand Name : " + vobj1.getName() + " Mileage : "
+ bikeVal + " km/l");
        System.out.println("Brand Name : " + vobj3.getName() + " Mileage : "
+ scootVal + " km/l");
    }
}
```